UNIT2

1 .What is Software Project Management Complexities?

Software Project Management Complexities refer to the various difficulties to manage a software project. It recognizes in many different ways. The main goal of software project management is to enable a group of developers to work effectively toward the successful completion of a project in a given time. But software project management is a very difficult task.

Earlier many projects have failed due to faulty project management practices. Management of software projects is much more complex than management of many other types of projects. In this article, we will discuss the types of Complexity as well as the factors that make Project Management Complex.

Types of Complexity

The following are the types of complexity in software project management:

- **Time Management Complexity:** Complexities to estimate the duration of the project. It also includes the complexities to make the schedule for different activities and timely completion of the project.
- **Cost Management Complexity:** Estimating the total cost of the project is a very difficult task and another thing is to keep an eye that the project does not overrun the budget.
- **Quality Management Complexity:** The quality of the project must satisfy the customer's requirements. It must assure that the requirements of the customer are fulfilled.
- **Risk Management Complexity:** Risks are the unanticipated things that may occur during any phase of the project. Various difficulties may occur to identify these risks and make amendment plans to reduce the effects of these risks.
- **Human Resources Management Complexity:** It includes all the difficulties regarding organizing, managing, and leading the project team.
- **Communication Management Complexity:** All the members must interact with all the other members and there must be good communication with the customer.
- **Infrastructure complexity:** Computing infrastructure refers to all of the operations performed on the devices that execute our code. Networking, load balancers, queues, firewalls, security, monitoring, databases, shading, etc. We are solely interested in dealing with data, processing business policy rules, and clients since we are software engineers that are committed to providing value in a continuous stream. The aforementioned infrastructure ideas are nothing more than irksome minutiae that don't offer any benefit to the clients. Since it is a necessary evil, we view infrastructure as accidental complexity. Our policies for scaling, monitoring, and other issues are of little interest to our paying clients.
- **Deployment complexity:** A release candidate, or finalized code, has to be synchronized from one system to another. Conceptually, such an operation ought to be simple. To perform this synchronization swiftly and securely in practice proves to be difficult.
- **API complexity:** An API should ideally not be any more difficult to use than calling a function. However, that hardly ever occurs. These calls are inadvertently complicated due to authentication, rate restrictions, retries, mistakes, and other factors.
- **Procurement Management Complexity:** Projects need many services from third parties to complete the task. These may increase the complexity of the project to acquire the services.

- **Integration Management Complexity:** The difficulties regarding coordinating processes and developing a proper project plan. Many changes may occur during the project development and it may hamper the project completion, which increases the complexity.
    - **Invisibility:** Until the development of a software project is complete, Software remains invisible. Anything that is invisible, is difficult to manage and control. Software project managers cannot view the progress of the project due to the invisibility of the software until it is completely developed. The project manager can monitor the modules of the software that have been completed by the development team and the documents that have been prepared, which are rough indicators of the progress achieved. Thus invisibility causes a major problem in the complexity of managing a software project.
    - **Changeability:** Requirements of a software product are undergone various changes. Most of these change requirements come from the customer during the software development. Sometimes these change requests resulted in redoing of some work, which may cause various risks and increase expenses. Thus frequent changes to the requirements play a major role to make software project management complex.
    - **Interaction:** Even moderate-sized software has millions of parts (functions) that interact with each other in many ways such as data coupling, serial and concurrent runs, state transitions, control dependency, file sharing, etc. Due to the inherent complexity of the functioning of a software product in terms of the basic parts making up the software, many types of risks are associated with its development. This makes managing software projects much more difficult compared to many other kinds of projects.
    - **Uniqueness:** Every software project is usually associated with many unique features or situations. This makes every software product much different from the other software projects. This is unlike the projects in other domains such as building construction, bridge construction, etc. where the projects are more predictable. Due to this uniqueness of the software projects, during the software development, a project manager faces many unknown problems that are quite dissimilar to other software projects that he had encountered in the past. As a result, a software project manager has to confront many unanticipated issues in almost every project that he manages.
    - **The exactness of the Solution:** A small error can create a huge problem in a software project. The solution must be exact according to its design. The parameters of a function call in a program are required to be correct with the function definition. This requirement of exact conformity of the parameters of a function introduces additional risks and increases the complexity of managing software projects.
    - **Team-oriented and Intellect-intensive work:** Software development projects are team-oriented and intellect-intensive work. The software cannot be developed without interaction between developers. In a software development project, the life cycle activities are not only intellect-intensive, but each member has to typically interact, review the work done by other members, and interface with several other team members creating various complexity to manage software projects.
    - **The huge task regarding Estimation:** One of the most important aspects of software project management is Estimation. During project planning, a project manager has to estimate the cost of the project, the probable duration to complete

the project, and how much effort is needed to complete the project based on size estimation. This estimation is a very complex task, which increases the complexity of software project management.

2. Role and Responsibilities of a software Project Manager – Software Engineering

A software project manager is the most important person inside a team who takes the overall responsibilities to manage the software projects and plays an important role in the successful completion of the projects. This article focuses on discussing the role and responsibilities of a software project manager.

Table of Content

- Who is a Project Manager?
- Project Planning
- Objectives of Project Planning
- Activities Performed by Project Manager
- Features of a Good Project Manager

Who is a Project Manager?

A project manager has to face many difficult situations to accomplish these works. The job responsibilities of a project manager range from invisible activities like building up team morale to highly visible customer presentations. Most of the managers take responsibility for writing the project proposal, project cost estimation, scheduling, project staffing, software process tailoring, project monitoring and control, software configuration management, risk management, managerial report writing, and presentation, and interfacing with clients.

The tasks of a project manager are classified into two major types:

1. Project planning
2. Project monitoring and control

Project Planning

Project planning is undertaken immediately after the feasibility study phase and before the starting of the requirement analysis and specification phase. Once a project is feasible, Software project managers start project planning. Project planning is completed before any development phase starts.

1. Project planning involves estimating several characteristics of a project and then plan the project activities based on these estimations.
2. Project planning is done with most care and attention.
3. A wrong estimation can result in schedule slippage.

4. Schedule delay can cause customer dissatisfaction, which may lead to a project failure.
5. Before starting a software project, it is essential to determine the tasks to be performed and properly manage allocation of tasks among individuals involved is the software development.
6. Hence, planning is important as it results in effective software development.
7. Project planning is an organized and integrated management process, which focuses on activities required for successful completion of the project.
8. It prevents obstacles that arise in the project such as changes in projects or organizations objectives, non-availability of resources, and so on.
9. Project planning also helps in better utilization of resources and optimal usage of the allotted time for a project.
10. For effective project planning, in addition to a very good knowledge of various estimation techniques, experience is also very important.

Objectives of Project Planning

1. It defines the roles and responsibilities of the project management team members .
2. It ensures that the project management team works according to the business objectives.
3. It checks feasibility of the schedule and user requirements.
4. It determines project constraints, several individuals help in planning the project.

Activities Performed by Project Manager

1. Project Estimation

Project Size Estimation is the most significant parameter based on which all other estimations like cost, duration and effort are made.

- **Cost Estimation:** Total expenses to develop the software product is estimated.
- **Time Estimation:** The total time required to complete the project.
- **Effort Estimation:** The effort needed to complete the project is estimated.

2. Scheduling

After the completion of the estimation of all the project parameters, scheduling for manpower and other resources is done.

3. Staffing

Team structure and staffing plans are made.

4. Risk Management

The project manager should identify the unanticipated risks that may occur during project development risk, analyze the damage that might cause these risks, and take a risk reduction plan to cope with these risks.

5. Miscellaneous Plans

This includes making several other plans such as quality assurance plans, configuration management plans, etc.

- **Lead the team:** The project manager must be a good leader who makes a team of different members of various skills and can complete their individual tasks.
- **Motivate the team-member:** One of the key roles of a software project manager is to encourage team members to work properly for the successful completion of the project.

- **Tracking the progress:** The project manager should keep an eye on the progress of the project. A project manager must track whether the project is going as per plan or not. If any problem arises, then take the necessary action to solve the problem. Moreover, check whether the product is developed by maintaining correct coding standards or not.
- **Liaison:** The project manager is the link between the development team and the customer. Project manager analysis the customer requirements and convey it to the development team and keep telling the progress of the project to the customer. Moreover, the project manager checks whether the project is fulfilling the customer's requirements or not.
- **Monitoring and reviewing:** Project monitoring is a continuous process that lasts the whole time a product is being developed, during which the project manager compares actual progress and cost reports with anticipated reports as soon as possible. While most firms have a formal system in place to track progress, qualified project managers may still gain a good understanding of the project's development by simply talking with participants.
- **Documenting project report:** The project manager prepares the documentation of the project for future purposes. The reports contain detailed features of the product and various techniques. These reports help to maintain and enhance the quality of the project in the future.
- **Reporting:** Reporting project status to the customer and his or her organization is the responsibility of the project manager. Additionally, they could be required to prepare brief, well-organized pieces that summarize key details from in-depth studies.

Features of a Good Project Manager

1. Knowledge of project estimation techniques.
2. Good decision-making abilities at the right time.
3. Previous experience managing a similar type of projects.
4. Good communication skills to meet the customer satisfaction.
5. A project manager must encourage all the team members to successfully develop the product.
6. He must know the various type of risks that may occur and the solution to these problems.

3. Metrics for Project Size Estimation

Estimation of the size of the software is an essential part of Software Project Management. It helps the project manager to further predict the effort and time that will be needed to build the project. Here are some of the measures that are used in project size estimation:

1. Lines of Code (LOC)

As the name suggests, LOC counts the total number of lines of source code in a project. The units of LOC are:

1. **KLOC:** Thousand lines of code
2. **NLOC:** Non-comment lines of code
3. **KDSI:** Thousands of delivered source instruction

- The size is estimated by comparing it with the existing systems of the same kind. The experts use it to predict the required size of various components of software and then add them to get the total size.
- It's tough to estimate LOC by analyzing the problem definition. Only after the whole code has been developed can accurate LOC be estimated. This statistic is of little utility to project

managers because project planning must be completed before development activity can begin.

- Two separate source files having a similar number of lines may not require the same effort. A file with complicated logic would take longer to create than one with simple logic. Proper estimation may not be attainable based on LOC.
- The length of time it takes to solve an issue is measured in LOC. This statistic will differ greatly from one programmer to the next. A seasoned programmer can write the same logic in fewer lines than a newbie coder.

Advantages:

1. Universally accepted and is used in many models like COCOMO.
2. Estimation is closer to the developer's perspective.
3. Both people throughout the world utilize and accept it.
4. At project completion, LOC is easily quantified.
5. It has a specific connection to the result.
6. Simple to use.

Disadvantages:

1. Different programming languages contain a different number of lines.
2. No proper industry standard exists for this technique.
3. It is difficult to estimate the size using this technique in the early stages of the project.
4. When platforms and languages are different, LOC cannot be used to normalize.

2. Number of Entities in ER Diagram

ER model provides a static view of the project. It describes the entities and their relationships. The number of entities in the ER model can be used to measure the estimation of the size of the project. The number of entities depends on the size of the project. This is because more entities needed more classes/structures thus leading to more coding.

Advantages:

1. Size estimation can be done during the initial stages of planning.
2. The number of entities is independent of the programming technologies used.

Disadvantages:

1. No fixed standards exist. Some entities contribute more to project size than others.
2. Just like FPA, it is less used in the cost estimation model. Hence, it must be converted to LOC.

3. Total Number of Processes in DFD

Data Flow Diagram(DFD) represents the functional view of software. The model depicts the main processes/functions involved in software and the flow of data between them. Utilization of the number of functions in DFD to predict software size. Already existing processes of similar type are studied and used to estimate the size of the process. The sum of the estimated size of each process gives the final estimated size.

Advantages:

1. It is independent of the programming language.
2. Each major process can be decomposed into smaller processes. This will increase the accuracy of the estimation.

Disadvantages:

1. Studying similar kinds of processes to estimate size takes additional time and effort.
2. All software projects are not required for the construction of DFD.

4. Function Point Analysis

In this method, the number and type of functions supported by the software are utilized to find FPC(function point count). The steps in function point analysis are:

1. Count the number of functions of each proposed type.
2. Compute the Unadjusted Function Points(UFP).
3. Find the Total Degree of Influence(TDI).
4. Compute Value Adjustment Factor(VAF).
5. Find the Function Point Count(FPC).

The explanation of the above points is given below:

1. Count the number of functions of each proposed type:

Find the number of functions belonging to the following types:

- External Inputs: Functions related to data entering the system.
- External outputs: Functions related to data exiting the system.
- External Inquiries: They lead to data retrieval from the system but don't change the system.
- Internal Files: Logical files maintained within the system. Log files are not included here.
- External interface Files: These are logical files for other applications which are used by our system.

2. Compute the Unadjusted Function Points(UFP):

Categorize each of the five function types as simple, average, or complex based on their complexity. Multiply the count of each function type with its weighting factor and find the weighted sum. The weighting factors for each type based on their complexity are as follows:

Function type	Simple	Average	Complex
External Inputs	3	4	6
External Output	4	5	7
External Inquiries	3	4	6
Internal Logical Files	7	10	15
External Interface Files	5	7	10

3. Find the Total Degree of Influence:

Use the '14 general characteristics of a system to find the degree of influence of each of them. The sum of all 14 degrees of influence will give the TDI. The range of TDI is 0 to 70. The 14 general characteristics are: Data Communications, Distributed Data Processing, Performance, Heavily Used Configuration, Transaction Rate, On-Line Data Entry, End-user Efficiency, Online Update, Complex Processing Reusability, Installation Ease, Operational Ease, Multiple

Sites and Facilitate Change.
Each of the above characteristics is evaluated on a scale of 0-5.

4. Compute Value Adjustment Factor(VAF):

Use the following formula to calculate VAF:

*VAF = (TDI * 0.01) + 0.65*

5. Find the Function Point Count:

Use the following formula to calculate FPC:

*FPC = UFP * VAF*

Advantages:

1. It can be easily used in the early stages of project planning.
2. It is independent of the programming language.
3. It can be used to compare different projects even if they use different technologies(database, language, etc).

Disadvantages:

1. It is not good for real-time systems and embedded systems.
2. Many cost estimation models like COCOMO use LOC and hence FPC must be converted to LOC.

When Should Estimates Take Place?

Project size estimates must take place at multiple key points throughout the project lifecycle. It should take place during the following stages to ensure accuracy and relevance:

1. **Project Initiation:** Project is assessed to determine its feasibility and scope.
2. **Project Planning:** Precise estimates are done to create a realistic budget and timeline.
3. **Project Execution:** Res-estimation when there are significant changes in scope.
4. **Project Monitoring and Control:** Regular reviews to make sure that the project is on track.
5. **Project Closeout:** Comparing original estimates with actual outcomes and documenting estimation accuracy.

Challenges in Project Size Estimation

Project size estimation can be challenging due to multiple factors. Here are some factors that can affect the accuracy and reliability of estimates:

1. **Unclear Requirements:** Initial project requirements can be vague or subject to change, thus making it difficult to estimate accurately.
2. **Lack of Historical Data:** Without access to the data of similar past projects, it becomes difficult to make informed estimates, thus estimates becoming overly optimistic or pessimistic and leading to inaccurate planning.
3. **Interdependencies:** Project with numerous interdependent tasks are harder to estimate due to the complicated interactions between components.
4. **Productivity Variability:** Estimating the productivity of resources and their availability can be challenging due to fluctuations and uncertainties.
5. **Risks:** Identifying and quantifying risks and uncertainties is very difficult. Underestimating the potential risks can lead to inadequate contingency planning, thus causing the project to go off track.

Improving Accuracy in Project Size Estimation

Improving the accuracy of project size estimation involves a combination of techniques and best practices. Here are some key strategies to enhance estimation accuracy:

1. **Define Clear Requirements:** Ensure all project requirements are thoroughly documented and engage all stakeholders early and frequently to clarify and validate the requirements.
2. **Use Historical Data:** Use data from similar past projects to make informed estimates.
3. **Use Estimation Techniques:** Use various estimation techniques like Analogue Estimation, Parametric Estimation, Bottom-Up Estimation, and Three-Point Estimation.
4. **Break Down the Project:** Use Work Breakdown Structure (WBS) and detailed take analysis to make sure that each task is specific and measurable.
5. **Incorporate Expert Judgement:** Engage subject matter experts and experienced team members to provide input on estimates.

Future of Project Size Estimation

The future of project size estimation will be shaped by the advancements in technology and methodologies. Here are some key developments that can define the future of project size estimation:

1. **Smarter Technology:** Artificial intelligence (AI) could analyze past projects and code to give more accurate forecasts, considering how complex the project features are.
2. **Data-Driven Insights:** Instead of just lines of code, estimates could consider factors like the number of users, the type of software (mobile app vs. web app), and how much data it handles.
3. **Human-AI Collaboration:** Combining human expertise with AI can enhance the decision-making process in project size estimation.
4. **Collaborative Platforms:** Tools that facilitate collaboration among geographically dispersed teams can help to enhance the project size estimation process.
5. **Agile Methodologies:** The adoption of agile methodologies can promote continuous estimation and iterative refinement.

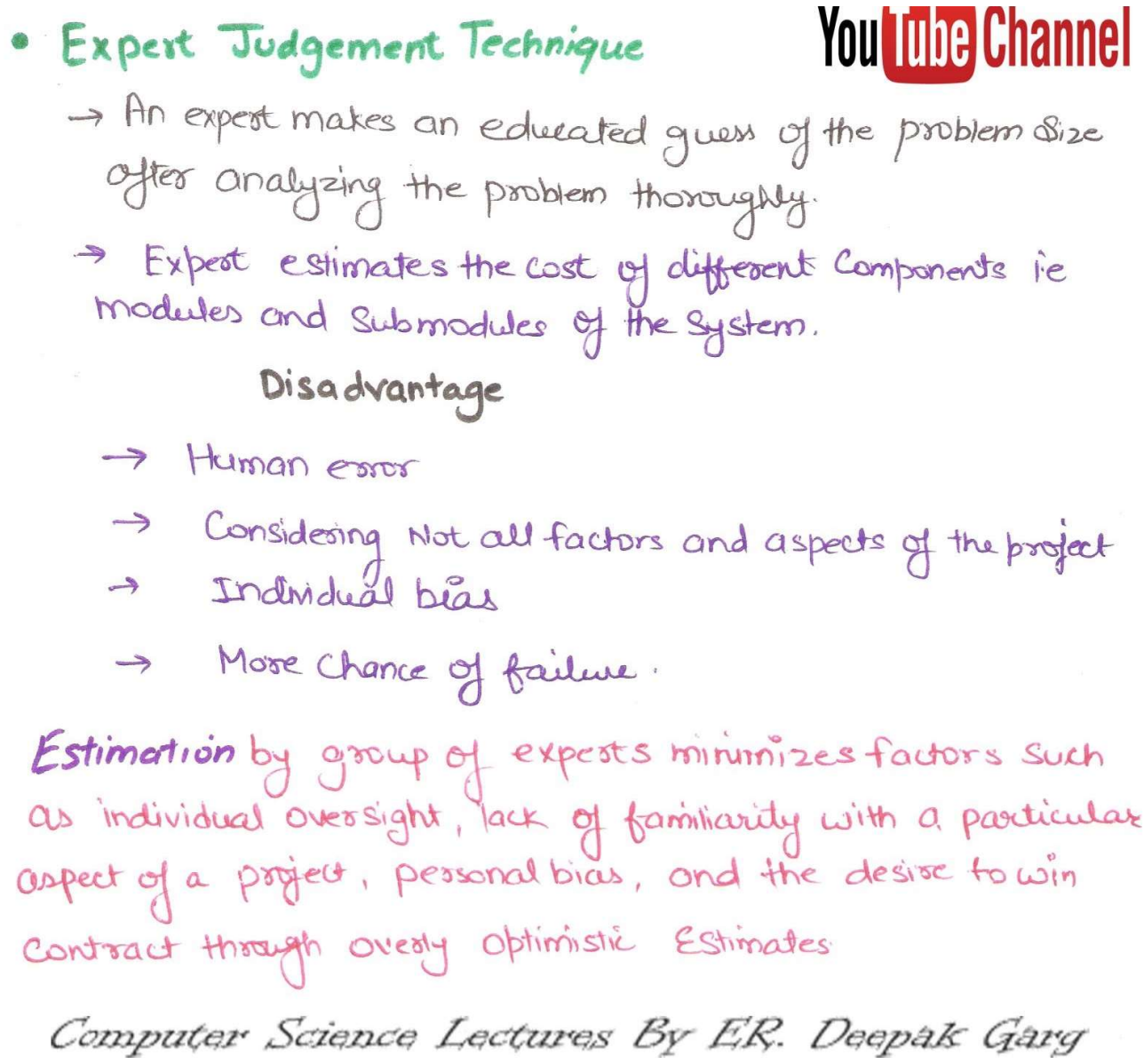
4. Project Estimation techniques

1. **Expert Judgment:** In this technique, a group of experts in the relevant field estimates the project size based on their experience and expertise. This technique is often used when there is limited information available about the project.
2. **Analogous Estimation:** This technique involves estimating the project size based on the similarities between the current project and previously completed projects. This technique is useful when historical data is available for similar projects.
3. **Bottom-up Estimation:** In this technique, the project is divided into smaller modules or tasks, and each task is estimated separately. The estimates are then aggregated to arrive at the overall project estimate.
4. **Three-point Estimation:** This technique involves estimating the project size using three values: optimistic, pessimistic, and most likely. These values are then used to calculate the expected project size using a formula such as the PERT formula.
5. **Function Points:** This technique involves estimating the project size based on the functionality provided by the software. Function points consider factors such as inputs, outputs, inquiries, and files to arrive at the project size estimate.
6. **Use Case Points:** This technique involves estimating the project size based on the number of use cases that the software must support. Use case points consider factors such as the complexity of each use case, the number of actors involved, and the number of use cases.
7. **Parametric Estimation:** For precise size estimation, mathematical models founded on project parameters and historical data are used.

8. **COCOMO (Constructive Cost Model):** It is an algorithmic model that estimates effort, time, and cost in software development projects by taking into account several different elements.
9. **Wideband Delphi:** Consensus-based estimating method for balanced size estimations that combines expert estimates from anonymous experts with cooperative conversations.
10. **Monte Carlo Simulation:** This technique, which works especially well for complicated and unpredictable projects, estimates project size and analyses hazards using statistical methods and random sampling.

Each of these techniques has its strengths and weaknesses, and the choice of technique depends on various factors such as the project's complexity, available data, and the expertise of the team.

5. Empirical Estimation technique



TutorialsSpace.com
A SIMPLE LEARNING
Subscribe to our YouTube Channel

- **Delphi Cost Estimation:-**

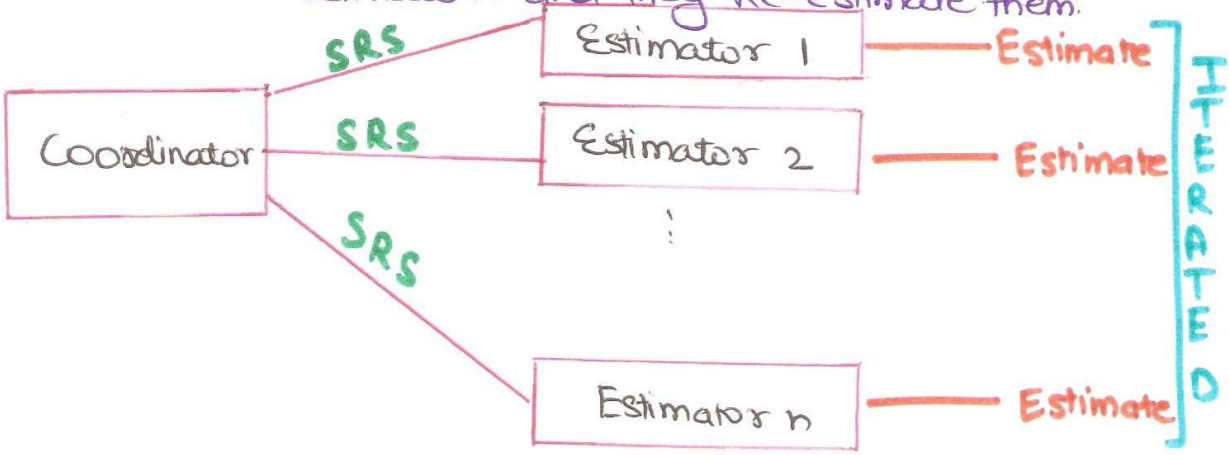
Estimation By → Experts Group + Coordinators

Role of Members:

Coordinator provide a copy of Software Requirement Specification (SRS) document and a form of Recording his Cost estimate to each estimator

Estimators completes their individual estimates anonymously and Submit to the coordinator with mentioning, if any, unusal Characteristic of product which has influenced his estimation.

The Coordinator prepare and distibutes the Summary of the responses to all estimators and they Re-estimate them.



This Process is ITERATED for Several Rounds.

No Discussion is allowed among the estimators during the entire Estimation process-because then may be many estimators get easily influenced by the rationale of

an estimator who may be more experienced or Senior

After the completion of several iterations of estimations, the coordinator takes the responsibility of compiling the results and preparing the final estimates.

Subscribe to our
YouTube Channel

TutorialsSpace.com
A SIMPLE LEARNING

6. Halstead's software science

Halstead's complexity measurement was developed to measure a program module's complexity directly from source code, with emphasis on computational complexity.

The Halstead's measures are based on four scalar number derived directly from a program's source code:

n1 is number of distinct operators.

n2 is number of distinct operands.

N1 is total number of distinct operators.

N2 is total number of distinct operands.

From these numbers, five measures are derived:

Measure	Symbol	Formula
Program length	N	N=N1+N2
Program vocabulary	N	n =n1+n2
Volume	V	V=N * (log2n)
Difficulty	D	D=(n1/ 2)* (N2/ 2)
Effort	E	E =D * V

Halstead's uses certain measures such as program length, program vocabulary, volume, difficulty, and effort for the given algorithm. By this Halstead's is trying to show that the program length can be calculated, volume of algorithm can be estimated. The above given table shows how actually these measure can be obtained.

The Halstead's measures are applicable to operational system and to development efforts once the code has been written. Thus using Halstead's measurement experimental verification can be performed in software science.

Program length:

The length of a program is total usage of operators and operands in the program.

Length (N)=N1+N2

Program vocabulary:

The Program vocabulary is the number of unique operators and operands used in the program.

Vocabulary (n)=n1+n2

Program Volume:

The Program Volume can be defined as minimum number of bits needed to encode the program.

Volume (V)= N log2n

Length estimation:

N=n1log2n1+n2log2n2

Guideline for calculating operands and operators:

1. All the variables and constants are considered as operands.
2. Local variables with same name, if occurring in different functions are counted as unique operand.
3. Function calls are considered as operators.
4. The looping statements, do…while, while, for, are operators. The statements if, If… else, are operators. The switch … case statements are considered as operators.
5. There serve worlds, returns, default, continue, break, size of are all operators.
6. The brackets, commas, semicolons, are operators.
7. The unary and binary operators are considered as operators. The & is considered as operator.
8. In arrays, array name and index a reconsidered as operands and[] is considered as operator.
9. All hash directive scan be ignored.
10. Comments are not considered.
11. In Goto statement, goto is considered as operator and label as operand.
12. The looping statements, do…while, while, for, are operators. The statements if, If… else, are operators. The switch … case statements are considered as operators.
13. There serve worlds, returns, default, continue, break, size of are all operators.
14. The brackets, commas, semicolons, are operators.
15. The unary and binary operators are considered as operators. The & is considered as operator.
16. In arrays, array name and index a reconsidered as operands and[] is considered as operator.
17. All hash directive scan be ignored.
18. Comments are not considered.
19. In Goto statement, goto is considered as operator and label as operand.

Example: Obtain Halstead's length and volume measure for following C function. Void swap (int a[ ], int i)

{

int temp; Temp = a[i]; a[i]

= a[i+1]; a[i+1]=temp;

}

Example: Obtain Halstead's length and volume measure for following C

function. Void swap (int a[ ], int i)

{

int temp;

Temp = a[i];

a[i] = a[i+1];

a[i+1]=temp;

}

We first find out the operands and operators from above function along with the iroccurrences.

Operands	Occurrences	Operators	Occurrences
swap	1	( )	1
a	5	{ }	1
i	5	void	1
temp	3	int	3
1	2	[ ]	5
		,	1

		;	4
		=	3
		+	2
n1=5	N1=16	n2=9	N2=21

N= N1+ N2

=16 +21 =37

N = 37

n = n1+n2

=5 +9 =14

n = 14

Estimated length= n1logn1+ n2logn2

=5 log 5 + 9 log 9

=5 * 2.32 +9 * 2.19 =31.37

Estimated length=31.37 Volume

= N * log n =37 * log (14)

37 * 2.63 =97.64

Volume (V) = 97.64

7. Risk Management

"Tomorrow problems are today's risk." Hence, a clear definition of a "risk" is a problem that could cause some loss or threaten the progress of the project, but which has not happened yet.

These potential issues might harm cost, schedule or technical success of the project and the quality of our software device, or project team morale.

Risk Management is the system of identifying addressing and eliminating these problems before they can damage the project.

We need to differentiate risks, as potential issues, from the current problems of the project.

Different methods are required to address these two kinds of issues.

For example, staff storage, because we have not been able to select people with the right technical skills is a current problem, but the threat of our technical persons being hired away by the competition is a risk.

Risk Management

A software project can be concerned with a large variety of risks. In order to be adept to systematically identify the significant risks which might affect a software project, it is essential to classify risks into different classes. The project manager can then check which risks from each class are relevant to the project.

There are three main classifications of risks which can affect a software project:

1. Project risks
2. Technical risks
3. Business risks

1. Project risks: Project risks concern differ forms of budgetary, schedule, personnel, resource, and customer-related problems. A vital project risk is schedule slippage. Since the software is intangible, it is very tough to monitor and control a software project. It is very tough to control something which cannot be identified. For any manufacturing program, such as the manufacturing of cars, the plan executive can recognize the product taking shape.

2. Technical risks: Technical risks concern potential method, implementation, interfacing, testing, and maintenance issue. It also consists of an ambiguous specification, incomplete specification, changing specification, technical

uncertainty, and technical obsolescence. Most technical risks appear due to the development team's insufficient knowledge about the project.

3. Business risks: This type of risks contain risks of building an excellent product that no one need, losing budgetary or personnel commitments, etc.

Other risk categories

1. **1. Known risks:** Those risks that can be uncovered after careful assessment of the project program, the business and technical environment in which the plan is being developed, and more reliable data sources (e.g., unrealistic delivery date)
2. **2. Predictable risks:** Those risks that are hypothesized from previous project experience (e.g., past turnover)
3. **3. Unpredictable risks:** Those risks that can and do occur, but are extremely tough to identify in advance.

Principle of Risk Management

1. **Global Perspective:** In this, we review the bigger system description, design, and implementation. We look at the chance and the impact the risk is going to have.
2. **Take a forward-looking view:** Consider the threat which may appear in the future and create future plans for directing the next events.
3. **Open Communication:** This is to allow the free flow of communications between the client and the team members so that they have certainty about the risks.
4. **Integrated management:** In this method risk management is made an integral part of project management.
5. **Continuous process:** In this phase, the risks are tracked continuously throughout the risk management paradigm.

6. Risk Management Activities

Risk management consists of three main activities, as shown in fig:



Risk Assessment

The objective of risk assessment is to division the risks in the condition of their loss, causing potential. For risk assessment, first, every risk should be rated in two methods:

- The possibility of a risk coming true (denoted as r).
- The consequence of the issues relates to that risk (denoted as s).

Based on these two methods, the priority of each risk can be estimated:

$$p = r * s$$

Where p is the priority with which the risk must be controlled, r is the probability of the risk becoming true, and s is the severity of loss caused due to the risk becoming true. If all identified risks are set up, then the most likely and damaging

risks can be controlled first, and more comprehensive risk abatement methods can be designed for these risks.

1. Risk Identification: The project organizer needs to anticipate the risk in the project as early as possible so that the impact of risk can be reduced by making effective risk management planning.

A project can be of use by a large variety of risk. To identify the significant risk, this might affect a project. It is necessary to categories into the different risk of classes.

There are different types of risks which can affect a software project:

1. **Technology risks:** Risks that assume from the software or hardware technologies that are used to develop the system.
2. **People risks:** Risks that are connected with the person in the development team.
3. **Organizational risks:** Risks that assume from the organizational environment where the software is being developed.
4. **Tools risks:** Risks that assume from the software tools and other support software used to create the system.
5. **Requirement risks:** Risks that assume from the changes to the customer requirement and the process of managing the requirements change.
6. **Estimation risks:** Risks that assume from the management estimates of the resources required to build the system

2. Risk Analysis: During the risk analysis process, you have to consider every identified risk and make a perception of the probability and seriousness of that risk.

There is no simple way to do this. You have to rely on your perception and experience of previous projects and the problems that arise in them.

It is not possible to make an exact, the numerical estimate of the probability and seriousness of each risk. Instead, you should authorize the risk to one of several bands:

1. The probability of the risk might be determined as very low (0-10%), low (10-25%), moderate (25-50%), high (50-75%) or very high (+75%).
2. The effect of the risk might be determined as catastrophic (threaten the survival of the plan), serious (would cause significant delays), tolerable (delays are within allowed contingency), or insignificant.

Risk Control

It is the process of managing risks to achieve desired outcomes. After all, the identified risks of a plan are determined; the project must be made to include the most harmful and the most likely risks. Different risks need different containment methods. In fact, most risks need ingenuity on the part of the project manager in tackling the risk.

There are three main methods to plan for risk management:

1. **Avoid the risk:** This may take several ways such as discussing with the client to change the requirements to decrease the scope of the work, giving incentives to the engineers to avoid the risk of human resources turnover, etc.
2. **Transfer the risk:** This method involves getting the risky element developed by a third party, buying insurance cover, etc.
3. **Risk reduction:** This means planning method to include the loss due to risk. For instance, if there is a risk that some key personnel might leave, new recruitment can be planned.

Risk Leverage: To choose between the various methods of handling risk, the project plan must consider the amount of controlling the risk and the corresponding reduction of risk. For this, the risk leverage of the various risks can be estimated.

Risk leverage is the variation in risk exposure divided by the amount of reducing the risk.

Risk leverage = (risk exposure before reduction - risk exposure after reduction) / (cost of reduction)

1. Risk planning: The risk planning method considers each of the key risks that have b For each of the risks, you have to think of the behavior that you may take to minimize the disruption to the plan if the issue identified in the risk occurs.

You also should think about data that you might need to collect while monitoring the plan so that issues can be anticipated.

Again, there is no easy process that can be followed for contingency planning. It rely on the judgment and experience of the project manager.

2. Risk Monitoring: Risk monitoring is the method king that your assumption about the product, process, and business risks has not changed.

REQUIREMENT ANALYSIS AND SPECIFICATIONS

1. Requirement gathering and analysis:-

Requirements analysis or requirements engineering is a process used to determine the needs and expectations of a new product. It involves frequent communication with the stakeholders and end-users of the product to define expectations, resolve conflicts, and document all the key requirements

What is Requirements Gathering?

Requirements gathering are a crucial phase in the software development life cycle (SDLC) and project management. It involves collecting, documenting, and managing the requirements that define the features and functionalities of a system or application. The success of a project often depends on the accuracy and completeness of the gathered requirements in software.

Main Requirements Gathering Sub processes:

Requirements gathering are a critical phase in the software development lifecycle, and it involves several sub processes to ensure a comprehensive understanding of the project's needs. The main sub processes include:

Stakeholder Identification:

- **Objective:** Identify all stakeholders who will be affected by the system, directly or indirectly.
- **Process**: Conduct interviews, surveys, or workshops to determine the key individuals or groups involved.

Stakeholder Analysis:

- **Objective:** Understand the needs, expectations, and influence of each stakeholder.
- **Process:** Analyze stakeholder inputs to prioritize requirements and manage conflicting interests.

Problem Definition:

- **Objective:** Clearly define the problems or opportunities that the software system aims to address.
- **Process:** Engage stakeholders in discussions to uncover and articulate the core problems or opportunities.

Requirements Extraction:

- **Objective:** Gather detailed requirements by interacting with stakeholders.
- **Process:** Employ techniques such as interviews, surveys, observations, or brainstorming sessions to extract requirements.
-

Requirements Documentation:

- **Objective:** Document gathered requirements in a structured format.
- **Process:** Create requirements documents, use cases, user stories, or prototypes to capture and communicate requirements effectively.

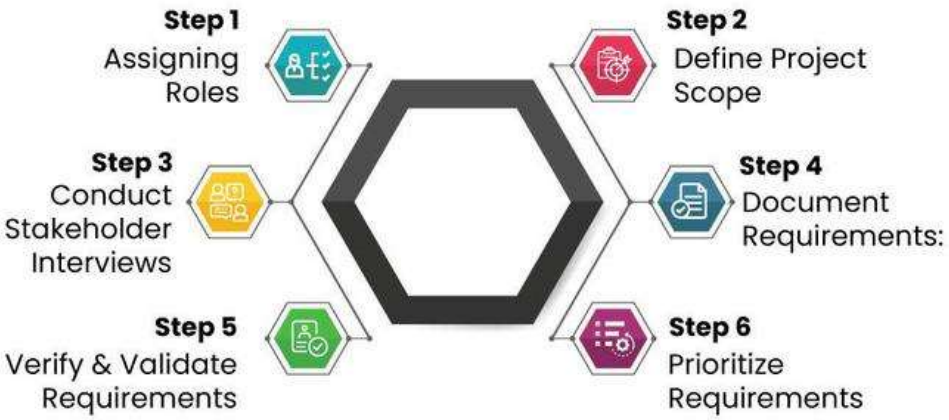
Validation and Verification:

- **Objective:** Ensure that gathered requirements are accurate, complete, and consistent.

Process: Conduct reviews, walkthroughs, or use validation tools to verify that the requirements meet the defined criteria

Processes of Requirements Gathering in Software Development:

There are 6 steps crucial for requirement gathering processes



Processes of Requirements Gathering in Software Development

Step 1- Assigning roles:

- The first step is to identify and engage with all relevant stakeholders. Stakeholders can include end-users, clients, project managers, subject matter experts, and anyone else who has a vested interest in the software project. Understanding their perspectives is essential for capturing diverse requirements.

Step 2- Define Project Scope:

- Clearly define the scope of the project by outlining its objectives, boundaries, and limitations. This step helps in establishing a common understanding of what the software is expected to achieve and what functionalities it should include.

Step 3- Conduct Stakeholder Interviews:

- Schedule interviews with key stakeholders to gather information about their needs, preferences, and expectations. Through open-ended questions and discussions, aim to uncover both explicit and implicit requirements. These interviews provide valuable insights that contribute to a more holistic understanding of the project.

Step 4- Document Requirements:

- Systematically document the gathered requirements. This documentation can take various forms, such as user stories, use cases, or formal specifications. Clearly articulate functional requirements (what the system should do) and non-functional requirements (qualities the system should have, such as performance or security).

Step 5- Verify and Validate Requirements:

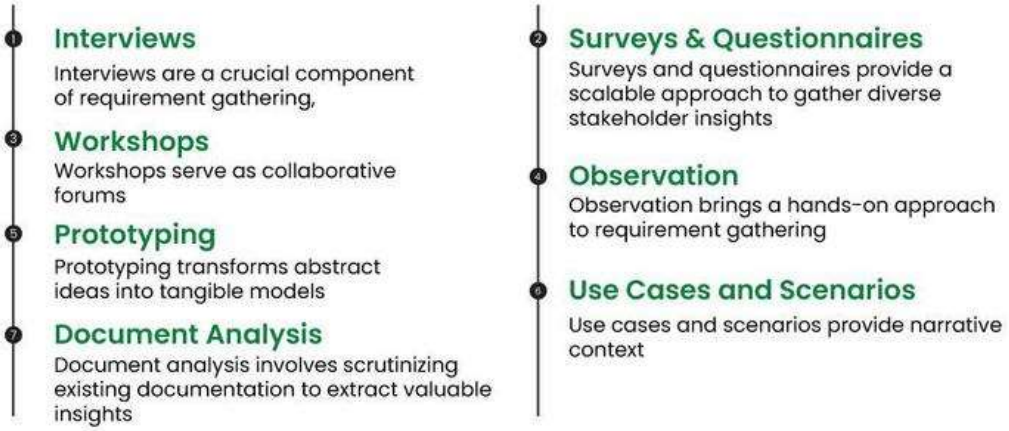
- Once the requirements are documented, it's crucial to verify and validate them. Verification ensures that the requirements align with the stakeholders' intentions, while validation ensures that the documented requirements will meet the project's goals. This step often involves feedback loops and discussions with stakeholders to refine and clarify requirements.

Step 6- Prioritize Requirements:

- Prioritize the requirements based on their importance to the project goals and constraints. This step helps in creating a roadmap for development, guiding the team on which features to prioritize. Prioritization is essential, especially when resources and time are limited.

Requirement Gathering Techniques:

Effective requirement gathering is essential for the success of a software development project. Various techniques are employed to collect, analyze, and document requirements



Requirement Gathering Techniques

Here are some commonly used requirement gathering techniques:

1. **Interviews:**
   - Conducting one-on-one or group interviews with stakeholders, including end-users, clients, and subject matter experts. This allows for direct interaction to gather detailed information about their needs, expectations, and concerns.
2. **Surveys and Questionnaires:**
   - Distributing surveys and questionnaires to a broad audience to collect information on a larger scale. This technique is useful for gathering feedback from a diverse set of stakeholders and can be particularly effective in large projects.
3. **Workshops:**
   - Organizing facilitated group sessions or workshops where stakeholders come together to discuss and define requirements. Workshops encourage collaboration, idea generation, and the resolution of conflicting viewpoints in a structured environment.
4. **Observation:**
   - Directly observing end-users in their work environment to understand their workflows, pain points, and preferences. Observational techniques help in uncovering implicit requirements that users might not explicitly state.
5. **Prototyping:**
   - Creating mockups or prototypes of the software to provide stakeholders with a tangible representation of the proposed system. Prototyping allows for early visualization and feedback, helping to refine requirements based on stakeholders' reactions.

6. **Use Cases and Scenarios:**

- Developing use cases and scenarios to describe how the system will be used in different situations. This technique helps in understanding the interactions between users and the system, making it easier to identify and document functional requirements.

7. **Document Analysis:**
   - Reviewing existing documentation, such as business process manuals, reports, and forms, to extract relevant information. This technique provides insights into the current processes and helps identify areas for improvement.

Why Requirement Gathering is important?

Requirement gathering holds immense importance in software development for several critical reasons:

1. **Clarity of Project Objectives:**
   - Requirement gathering sets the stage by defining and clarifying the objectives of the software project. It ensures that all stakeholders, including clients, users, and development teams, have a shared understanding of what needs to be achieved.
2. **Customer Satisfaction:**
   - Understanding and meeting customer needs is paramount for customer satisfaction. Requirement gathering allows developers to comprehend the expectations of end-users and clients, leading to the creation of a product that aligns with their desires and requirements.
3. **Scope Definition:**
   - Clearly defined requirements help in establishing the scope of the project. This delineation is crucial for managing expectations, avoiding scope creep (uncontrolled changes to project scope), and ensuring that the project stays on track.
4. **Reduced Misunderstandings:**
   - Ambiguities and misunderstandings are common sources of project failures. Requirement gathering facilitates clear communication between stakeholders, reducing the risk of misinterpretations and ensuring that everyone involved is on the same page.
5. **Risk Mitigation:**
   - Identifying and addressing potential issues at the requirements stage helps mitigate risks early in the development process. This proactive approach minimizes the chances of costly errors, rework, and delays later in the project life cycle.

Benefits of Requirements Gathering:

The benefits of effective requirements gathering in software development include:

- **Cost Reduction:** One of the primary benefits of effective requirements gathering is cost reduction. When requirements are well-defined and thoroughly understood at the beginning of a project, it minimizes the likelihood of costly changes and rework later in the development process.
- **Customer Satisfaction:** Clear and accurate requirements gathering directly contributes to customer satisfaction. When the end product aligns closely with the expectations and needs of the stakeholders, it enhances user experience and meets customer demands. This satisfaction is not only vital for the success of the current project but also contributes to positive relationships between the development team and clients, fostering trust and potential future collaborations.
- **Improved Communication:** Requirements gathering serves as a communication bridge between various stakeholders involved in a project, including developers, clients, users, and project managers. Miscommunication is a common source of project failures and delays. By clearly documenting and understanding requirements, the development team ensures that everyone involved has a shared vision of the project objectives, functionalities, and constraints.
- **Efficient Resource Utilization:** Thorough requirements gathering enables the efficient allocation and utilization of resources. Resources, including time, manpower, and technology, are finite and valuable. When requirements are well-defined, project teams can allocate resources more accurately, avoiding unnecessary expenditures or overcommitting resources to certain aspects of the project.
- **Enhanced Quality:** Well-documented requirements serve as the foundation for quality assurance throughout the development process. When the project team has a clear understanding of what needs to be achieved, they can establish quality standards and criteria from the outset. This clarity enables the implementation of effective testing strategies, ensuring that each aspect of the system is thoroughly evaluated against the specified requirements.
- **Risk Management:** Requirements gathering is a crucial component of effective risk management. By identifying potential risks early in the project, stakeholders can proactively address ambiguities, conflicting requirements, and other challenges that could pose a threat to the project's success.
- **Accurate Planning:** Accurate project planning is dependent on a clear understanding of project requirements. When requirements are well-documented, project managers can create realistic schedules, milestones, and deliverables. This accurate planning is crucial for setting expectations, managing stakeholder timelines, and ensuring that the project progresses according to the established timeline.

Common Obstacles in Software Requirements Gathering:

Common obstacles in software requirements gathering include:

- **Unclear Objectives:** Lack of clear project objectives can hinder requirements gathering. When stakeholders are unsure about what they want to achieve, it becomes challenging to define and prioritize requirements effectively. This can lead to confusion, scope creep, and difficulties in meeting project goals.
- **Ambiguous Requirements:** Ambiguities in requirements, such as vague language or conflicting statements, can create misunderstandings among stakeholders and the development team. Ambiguous requirements may result in deliverables that do not meet expectations and may require extensive rework.
- **Poor Stakeholder Involvement:** Insufficient involvement or engagement of key stakeholders can impede the requirements gathering process. When essential stakeholders are not actively participating or providing input, there is a risk of missing critical requirements or making decisions that do not align with the needs of the end-users.
- **Changing Requirements:** Requirements that undergo frequent changes during the development process, often referred to as "scope creep," can lead to project delays, increased costs, and challenges in maintaining project focus. It is essential to manage and control changes to prevent unnecessary disruptions.
- **Communication Barriers:** Communication challenges, such as language barriers, misinterpretations, or inadequate channels for information exchange, can hinder effective requirements gathering. It is crucial to establish clear communication channels and ensure that all stakeholders have a shared understanding of the terminology used in the project.
- **Overreliance on Documentation:** Depending solely on documentation without active collaboration and communication can lead to misunderstandings. Written requirements may not capture the complete context or evolving needs, making it essential to complement documentation with interactive processes like workshops and interviews.

Lack of User Involvement: Users are often the ultimate beneficiaries of the system, and their input is critical. Lack of user involvement or representation can result in systems that do not effectively meet their needs. It is important to actively involve end-users in the requirements gathering process to ensure the system's usability and acceptance

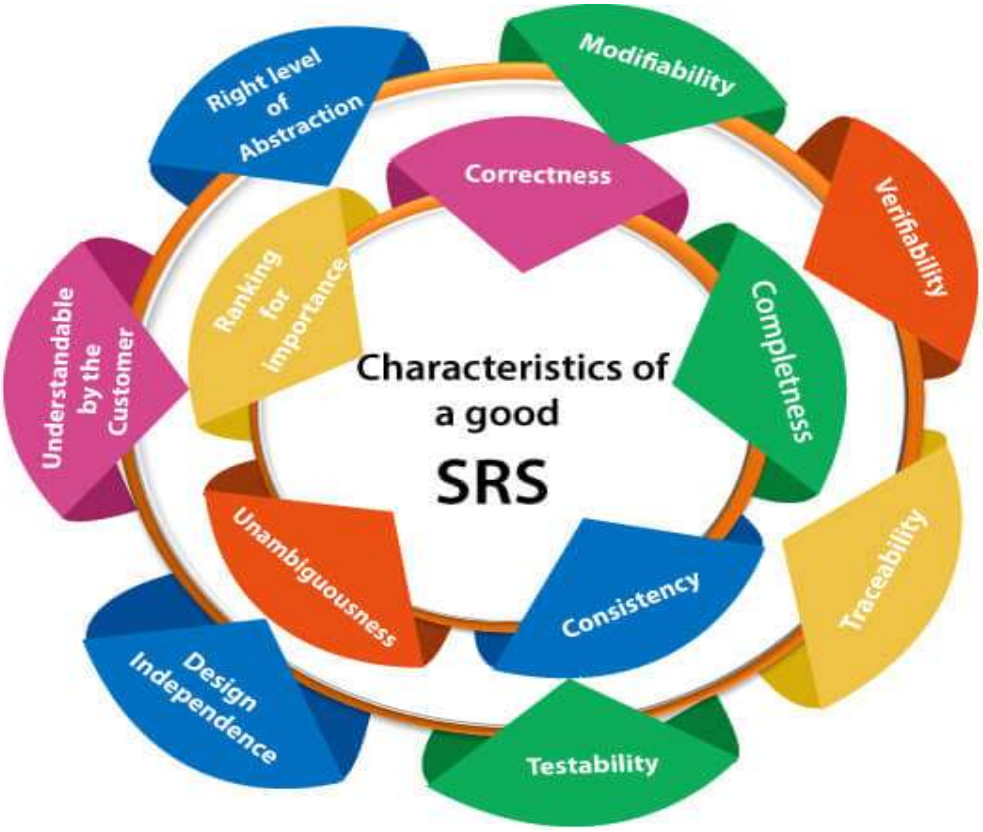
2. SOFTWARE RQUIREMENT SPECIFICATION (SRS)

Software Requirement Specifications

The production of the requirements stage of the software development process is **Software Requirements Specifications (SRS)** (also called a **requirements document**). This report lays a foundation for software engineering activities and is constructing when entire requirements are elicited and analyzed. **SRS** is a formal report, which acts as a representation of software that enables the customers to review whether it (SRS) is according to their requirements. Also, it comprises user requirements for a system as well as detailed specifications of the system requirements.

The SRS is a specification for a specific software product, program, or set of applications that perform particular functions in a specific environment. It serves several goals depending on who is writing it. First, the SRS could be written by the client of a system. Second, the SRS could be written by a developer of the system. The two methods create entirely various situations and establish different purposes for the document altogether. The first case, SRS, is used to define the needs and expectation of the users. The second case, SRS, is written for various purposes and serves as a contract document between customer and developer.

Characteristics of good SRS



Following are the features of a good SRS document:

1. Correctness: User review is used to provide the accuracy of requirements stated in the SRS. SRS is said to be perfect if it covers all the needs that are truly expected from the system.

2. Completeness: The SRS is complete if, and only if, it includes the following elements:

(1). All essential requirements, whether relating to functionality, performance, design, constraints, attributes, or external interfaces.

(2). Definition of their responses of the software to all realizable classes of input data in all available categories of situations.

3). Full labels and references to all figures, tables, and diagrams in the SRS and definitions of all terms and units of measure.

3. Consistency: The SRS is consistent if, and only if, no subset of individual requirements described in its conflict. There are three types of possible conflict in the SRS:

(1). The specified characteristics of real-world objects may conflicts. For example,

(a) The format of an output report may be described in one requirement as tabular but in another as textual.

(b) One condition may state that all lights shall be green while another states that all lights shall be blue.

(2). There may be a reasonable or temporal conflict between the two specified actions. For example,

(a) One requirement may determine that the program will add two inputs, and another may determine that the program will multiply them.

(b) One condition may state that "A" must always follow "B," while other requires that "A and B" co-occurs.

(3). Two or more requirements may define the same real-world object but use different terms for that object. For example, a program's request for user input may be called a "prompt" in one requirement's and a "cue" in another. The use of standard terminology and descriptions promotes consistency.

4. Unambiguousness: SRS is unambiguous when every fixed requirement has only one interpretation. This suggests that each element is uniquely interpreted. In case there is a method used with multiple definitions, the requirements report should determine the implications in the SRS so that it is clear and simple to understand.

5. Ranking for importance and stability: The SRS is ranked for importance and stability if each requirement in it has an identifier to indicate either the significance or stability of that particular requirement.

Typically, all requirements are not equally important. Some prerequisites may be essential, especially for life-critical applications, while others may be desirable. Each element should be identified to make these differences clear and explicit. Another way to rank requirements is to distinguish classes of items as essential, conditional, and optional.

6. Modifiability: SRS should be made as modifiable as likely and should be capable of quickly obtain changes to the system to some extent. Modifications should be perfectly indexed and cross-referenced.

7. Verifiability: SRS is correct when the specified requirements can be verified with a cost-effective system to check whether the final software meets those requirements. The requirements are verified with the help of reviews.

8. Traceability: The SRS is traceable if the origin of each of the requirements is clear and if it facilitates the referencing of each condition in future development or enhancement documentation.

There are two types of Traceability:

1. Backward Traceability: This depends upon each requirement explicitly referencing its source in earlier documents.

2. Forward Traceability: This depends upon each element in the SRS having a unique name or reference number.

The forward traceability of the SRS is especially crucial when the software product enters the operation and maintenance phase. As code and design document is modified, it is necessary to be able to ascertain the complete set of requirements that may be concerned by those modifications.

9. Design Independence: There should be an option to select from multiple design alternatives for the final system. More specifically, the SRS should not contain any implementation details.

10. Testability: An SRS should be written in such a method that it is simple to generate test cases and test plans from the report.

11. Understandable by the customer: An end user may be an expert in his/her explicit domain but might not be trained in computer science. Hence, the purpose of formal notations and symbols should be avoided too as much extent as possible. The language should be kept simple and clear.

12. The right level of abstraction: If the SRS is written for the requirements stage, the details should be explained explicitly. Whereas, for a feasibility study, fewer analysis can be used. Hence, the level of abstraction modifies according to the objective of the SRS.

Properties of a good SRS document

The essential properties of a good SRS document are the following:

Concise: The SRS report should be concise and at the same time, unambiguous, consistent, and complete. Verbose and irrelevant descriptions decrease readability and also increase error possibilities.

Structured: It should be well-structured. A well-structured document is simple to understand and modify. In practice, the SRS document

undergoes several revisions to cope up with the user requirements. Often, user requirements evolve over a period of time. Therefore, to make the modifications to the SRS document easy, it is vital to make the report well-structured.

Black-box view: It should only define what the system should do and refrain from stating how to do these. This means that the SRS document should define the external behavior of the system and not discuss the implementation issues. The SRS report should view the system to be developed as a black box and should define the externally visible behavior of the system. For this reason, the SRS report is also known as the black-box specification of a system.

Conceptual integrity: It should show conceptual integrity so that the reader can merely understand it. Response to undesired events: It should characterize acceptable responses to unwanted events. These are called system response to exceptional conditions.

Verifiable: All requirements of the system, as documented in the SRS document, should be correct. This means that it should be possible to decide whether or not requirements have been met in an implementation.

What is Fourth Generation Programming Language/Excutable Specifications?

A **Fourth Generation (Programming) Language (4GL)** is a grouping of programming languages that attempt to get closer than 3GLs to human language, a form of thinking, and conceptualization and are easier to use than 3GLs. It is a non-procedural language which means that the programmer defines what has to be done instead of how the task is to be completed.I4GL is more familiar and similar to human language. A compiler translates the whole program once i.e. it generates the object code for the program along with the list of errors. The execution is very fast. It allows users to develop software. These languages are usually designed for specific purposes and are commonly used in database programming and scripts such as PHP, Python, SQL, and many more. 4GLs make programming easier, more efficient, and more effective for users with less programming skills.

4th generation language is also known as a domain-specific language or a high-productivity language.

Components of Fourth Generation

The following are the components of 4GL:

- **Databases and Tables**: The Database and the tables on which the 4GL programs operate.
- **Form**: The screen that is displayed for the user data entry. The source code for forms is kept in operating system files with a .per suffix and contains instructions for how 4GL is to format the screen. For a form available to a 4GL program, it must be compiled into a file with a .frm suffix. The 4GL programs in turn reference the fields on the compiled screens.
- **Module**: Operating System files that contain the source code to your programs- a set of functions written in INFORMIX-4GL. These files have a suffix of .4gl and contain one or more components.
- **Main Function**: Each executable Informix program has the MAIN function; it is the first thing that is executed and in turn calls other functions.
- **Function**: Portions of 4GL programs that can be called from MAIN and other functions. These start with the FUNCTION keyword.
- **Reports**: Portions of 4GL programs that create reports. They include headers, groupings, sorting, and more. They start with the FUNCTION keyword.
- **Programs**: This is what is actually executed by the users. Depending on the 4GL product that you have (compiled or interpreted "Executable and Interpreted 4GL"), this file is executed either by the operating system or through a 4GL interpreter.

Features of 4GL

- It reduces programming costs and time.
- It is a high-level programming language.
- Its program has greater usability.
- It accesses the database.
- Minimum efforts from the user to obtain any information.

Types of 4GL

- Self-generator system.
- Report generator programming language.
- Form generators.
- Codeless programming.
- Data management.

Advantages of 4GL

- Smaller in size as compared to the previous generation's language.
- Graphics User Interface (GUI) technology was introduced.
- Low maintenance cost.
- The heat generated was negligible.
- Portable and cheaper than the previous generation.

Disadvantages of 4GL

- Requires complex structure.

- The latest technology is required for the manufacturing of Microprocessors.
- Less flexible than other languages.
- Memory consumption is high.

How does a 4GL differ from a 3GL?

4GLs, are high level languages that do not bother to seek how the solution is to be arrived at, but are more concerned with the solution It is unlike the 3GLs where you are provided with step by step on how to get to the solution.

Can 4GLs handle complex programming tasks?

4GLs are well suited for specific applications like database management and report generation yet they are not so effective where there is need to have closer control on the systems resources than with 3GLs. Such tasks, by their nature, may be accomplished better using lower level languages such as the 3GLs.

What are some examples of 4GLs?

Examples of 4GLs are SQL for Data Base, Python for Scripting and PHP for Web development. The idea here is that these languages are meant to be easy to use and as optimised for their purposes as is possible.

What is a Formal Technique?

A formal technique is a mathematical method to specify a hardware and/or software system, verify whether a specification is realisable, verify that an implementation satisfies its specification, prove properties of a system without necessarily running the system, etc. The mathematical basis of a formal method is provided by its specification language. More precisely, a formal specification language consists of two sets—syn and sem, and a relation sat between them. The set syn is called the syntactic domain, the set sem is called the semantic domain, and the relation sat is called the satisfaction relation. For a given specification syn, and model of the system sem, if sat (syn, sem), then syn is said to be the specification of sem, and sem is said to be the specificand of syn. The generally accepted paradigm for system development is through a hierarchy of abstractions. Each stage in this hierarchy is an implementation of its preceding stage and a specification of the succeeding stage. The different stages in this system development activity are requirements specification, functional design, architectural design, detailed design, coding, implementation, etc. In general, formal techniques can be used at every stage of the system development activity to verify that the output of one stage conforms to the output of the previous stage. Syntactic domains The syntactic domain of a formal specification language consists of an alphabet of symbols and a set of formation rules to construct well-formed formulas from the alphabet. The well-

formed formulas are used to specify a system. Semantic domains Formal techniques can have considerably different semantic domains. Abstract data type specification languages are used to specify algebras, theories, and programs. Programming languages are used to specify functions from input to output values. Concurrent and distributed system specification languages are used to specify state sequences, event sequences, state-transition sequences, synchronisation trees, partial orders, state machines, etc. Satisfaction relation Given the model of a system, it is important to determine whether an element of the semantic domain satisfies the specifications. This satisfaction is determined by using a homomorphism known as semantic abstraction function. The semantic abstraction function maps the elements of the semantic domain into equivalent classes. There can be different specifications describing different aspects of a system model, possibly using different specification languages. Some of these specifications describe the system's behaviour and the others describe the system's structure. Consequently, two broad classes of semantic abstraction functions are defined—those that preserve a system's behaviour and those that preserve a system's structure. Model versus property-oriented methods Formal methods are usually classified into two broad categories—the so-called model oriented and the property-oriented approaches. In a model-oriented style, one defines a system's behaviour directly by constructing a model of the system in terms of mathematical structures such as tuples, relations, functions, sets, sequences, etc. In the property-oriented style, the system's behaviour is defined indirectly by stating its properties, usually in the form of a set of axioms that the system must satisfy. Let us consider a simple producer/ consumer example. In a property-oriented style, we would probably start by listing the properties of the system like—the consumer can start consuming only after the producer has produced an item, the producer starts to produce an item only after the consumer has consumed the last item, etc. Two examples of property-oriented specification styles are axiomatic specification and algebraic specification. In a model-oriented style, we would start by defining the basic operations, p (produce) and c (consume). Then we can state that $S1 + p \Rightarrow S$, $S + c \Rightarrow S1$. Thus model-oriented approaches essentially specify a program by writing another, presumably simpler program. A few notable examples of popular model-oriented specification techniques are Z, CSP, CCS, etc. It is alleged that property-oriented approaches are more suitable for requirements specification, and that the model-oriented approaches are more suited to system design specification. The reason for this distinction is the fact that property-oriented approaches specify a system behaviour not by what they say of the system but by what they do not say of the system. Thus, property-oriented specifications permit a large number of possible implementations. Furthermore, property-oriented approaches specify a system by a conjunction of axioms, thereby making it easier to alter/augment specifications at a later stage. On the other hand, model-oriented methods do not support logical conjunctions and disjunctions, and thus even minor changes to a specification may lead to

overhauling an entire specification. Since the initial customer requirements undergo several changes as the development proceeds, the property-oriented style is generally preferred for requirements specification. Later in this chapter, we have discussed two property-oriented specification techniques.

AXIOMATIC SPECIFICATION

In axiomatic specification of a system, first-order logic is used to write the pre- and post- conditions to specify the operations of the system in the form of axioms. The pre-conditions basically capture the conditions that must be satisfied before an operation can successfully be invoked. In essence, the pre-conditions capture the requirements on the input parameters of a function. The post-conditions are the conditions that must be satisfied when a function completes execution for the function to be considered to have executed successfully. Thus, the post-conditions are essentially constraints on the results produced for the function execution to be considered successful. How to develop an axiomatic specifications? The following are the sequence of steps that can be followed to systematically develop the axiomatic specifications of a function: Establish the range of input values over which the function should behave correctly. Establish the constraints on the input parameters as a predicate. Specify a predicate defining the condition which must hold on the output of the function if it behaved properly. Establish the changes made to the function's input parameters after execution of the function. Pure mathematical functions do not change their input and therefore this type of assertion is not necessary for pure functions. Combine all of the above into pre- and post-conditions of the function.

ALGEBRAIC SPECIFICATION

In the algebraic specification technique, an object class or type is specified in terms of relationships existing between the operations defined on that type. It was first brought into prominence by Guttag [1980, 1985] in specification of abstract data types. Various notations of algebraic specifications have evolved, including those based on OBJ and Larch languages. Essentially, algebraic

specifications define a system as a heterogeneous algebra. A heterogeneous algebra is a collection of different sets on which several operations are defined. Traditional algebras are homogeneous. A homogeneous algebra consists of a single set and several operations defined in this set; e.g., {I, +, -, *, /}. In contrast, alphabetic strings S together with operations of concatenation and length {S, I, con, len}, is not a homogeneous algebra, since the range of the length operation is the set of integers. Each set of symbols in a heterogeneous algebra is called a sort of the algebra. To define a heterogeneous algebra, besides defining the sorts, we need to specify the involved operations, their signatures, and their domains and ranges. Using algebraic specification, we define the meaning of a set of interface procedures by using equations. An algebraic specification is usually presented in four sections. Types section: In this section, the sorts (or the data types) being used is specified. Exception section: This section gives the names of the exceptional conditions that might occur when different operations are carried out. These exception conditions are used in the later sections of an algebraic specification. Syntax section: This section defines the signatures of the interface procedures. The collection of sets that form input domain of an operator and the sort where the output is produced are called the signature of the operator. For example, PUSH takes a stack and an element as its input and returns a new stack that has been created. Equations section: This section gives a set of rewrite rules (or equations) defining the meaning of the interface procedures in terms of each other. In general, this section is allowed to contain conditional expressions.

By convention each equation is implicitly universally quantified over all possible values of the variables. This means that the equation holds for all possible values of the variable. Names not mentioned in the syntax section such r or e are variables. The first step in defining an algebraic specification is to identify the set of required operations. After having identified the required operators, it is helpful to classify them as either basic constructor operators, extra constructor operators, basic inspector operators, or extra inspection operators. The definition of these categories of operators is as follows: Basic construction operators: These operators are used to create or modify entities of a type. The basic construction operators are essential to generate all possible element of the type being specified. For example, ‘create’ and ‘append’ are basic construction operators in Example 4.13. Extra construction operators: These are the construction operators other than the basic construction operators. For example, the operator ‘remove’ in Example 4.13 is an extra construction operator, because even without using ‘remove’ it is possible to generate all values of the type being specified. Basic inspection operators: These operators evaluate attributes of a type without modifying them, e.g., eval, get, etc. Let S be the set of operators whose range is not the data type being specified—these are the inspection operators. The set of the basic operators S1 is a subset of S, such that each operator from S-S1 can be expressed in terms of the operators

from S1. Extra inspection operators: These are the inspection operators that are not basic inspectors. A simple way to determine whether an operator is a constructor (basic or extra) or an inspector (basic or extra) is to check the syntax expression for the operator. If the type being specified appears on the right hand side of the expression then it is a constructor, otherwise it is an inspection operator. For example, in Example 4.13, create is a constructor because point appears on the right hand side of the expression and point is the data type being specified. But, xcoord is an inspection operator since it does not modify the point type. A good rule of thumb while writing an algebraic specification, is to first establish which are the constructor (basic and extra) and inspection operators (basic and extra). Then write down an axiom for composition of each basic construction operator over each basic inspection operator and extra constructor operator. Also, write down an axiom for each of the extra inspector in terms of any of the basic inspectors. Thus, if there are m1 basic constructors, m2 extra constructors, n1 basic inspectors, and n2 extra inspectors, we should have $m1 \times (m2 + n1) + n2$ axioms. However, it should be clearly noted that these $m1 \times (m2 + n1) + n2$ axioms are the minimum required and many more axioms may be needed to make the specification complete. Using a complete set of rewrite rules, it is possible to simplify an arbitrary sequence of operations on the interface procedures. While developing the rewrite rules, different persons can come up with different sets of equations. However, while developing the equations one has to be careful that the equations should be able to handle all meaningful composition of operators, and they should have the unique termination and finite termination properties. These two properties of the rewrite rules are discussed later in this section.

EXAMPLE 4.13 Let us specify a data type point supporting the operations create, xcoord, ycoord, isequal; where the operations have their usual meaning.

Types:
defines point
uses Boolean, integer

Syntax:

1. create : integer × integer → point
2. xcoord : point → integer
3. ycoord : point → integer

4. isequal : point × point → boolean

Equations:

1. xcoord(create(x, y)) = x
2. ycoord(create(x, y)) = y
3. isequal(create(x1, y1), create(x2, y2)) = ((x1 = x2)and(y1 = y2))

In this example, we have only one basic constructor (create), and three basic inspectors (xcoord, ycoord, and isequal). Therefore, we have only 3 equations. The rewrite rules let you determine the meaning of any sequence of calls on the point type. Consider the following expression: isequal (create (xcoord (create(2, 3)), 5),create (ycoord (create(2, 3)), 5)). By applying the rewrite rule 1, you can simplify the given expression as isequal (create (2, 5), create (ycoord (create(2, 3)), 5)). By using rewrite rule 2, you can further simplify this as isequal (create (2, 5), create (3, 5)). This is false by rewrite rule 3.

Properties of algebraic specifications

Three important properties that every algebraic specification should possess are: Completeness: This property ensures that using the equations, it should be possible to reduce any arbitrary sequence of operations on the interface procedures. When the equations are not complete, at some step during the reduction process, we might not be able to reduce the expression arrived at that step by using any of the equations. There is no simple procedure to ensure that an algebraic specification is complete. Finite termination property: This property essentially addresses the following question: Do applications of the rewrite rules to arbitrary expressions involving the interface procedures always terminate? For arbitrary algebraic equations, convergence (finite termination) is undecidable. But, if the right hand side of each rewrite rule has fewer terms than the left, then the rewrite process must terminate. Unique termination property: This property indicates whether application of rewrite rules in different orders always result in the same answer. Essentially, to determine this property, the answer to the following question needs to be checked—Can all possible sequence of choices in application of the rewrite rules to an arbitrary expression involving the interface procedures always give the same answer? Checking the unique termination property is a very difficult problem.